```
            await Task.CompletedTask;
        }
    }
    // SaveToPickupDirectory method here
}
```

❶ The class implements the IEmailSender interface.

❷ Inject the IHostEnvironment interface in, so you can use it to generate a path for the email folder.

❸ Generate the folder path.

❹ Construct a message from the arguments sent in to the SendEmailAsync method.

❺ “Send” the email.

Take the code for the `SaveToPickupDirectory` method from the MailKit project’s FAQ, or from the download for this section (http://mng.bz/G18D), and insert it at the point indicated in the comment in listing 9.22.

NOTE This implementation is hardcoded to dump email messages into a folder on disk. While that is convenient when running the application locally, it is not much use when the application is live in production. In the final chapter, we will look at ways to manage this, so the behavior of the mailing service depends on the environment in which the application is running.

Now that you have an implementation of the `IEmailSender` service, you need to register your implementation with the service container:

```
builder.Services.AddTransient<IEmailSender, EmailService>();
```

Remember, by default, Identity does not require the user to confirm their account. The email service is only used if account confirmation is required, so you need to change the `SignIn` options in Program.cs. Add the bolded line shown in the following listing.

Listing 9.23 Making account confirmation required

```
builder.Services.AddDefaultIdentity<CityBreaksUser>(options => {
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireUppercase = false;
    options.SignIn.RequireConfirmedAccount = true;
}).AddEntityFrameworkStores<CityBreaksContext>();
```